

Product Brief

Companies that grow living cells in tanks, to produce drugs or food ingredients or materials, have to optimize the conditions those cells grow in. Temperature, feeding schedule, timing, oxygen, dozens of settings that all interact. Optimize it well and you get several times more product out of the same tank.

Finding this **global optimum** happens by running experiments. One experiment takes one to two weeks of physical work, capital and resources, and so they can't be cheaply re-run. A good team gets through maybe forty experiments in a year.

So imagine a lifetime budget of forty A/B tests. Two weeks each, no peeking at intermediate results, no rollback. You would stop caring about your analytics pipeline and start caring, obsessively, about which forty you picked.

Picking the experiments well is a solved mathematical problem (Experiment design) and has been around for decades. Using these techniques by wet-lab scientists however, is not very much mainstream. It needs someone who knows the statistics, data processing basics as well as modelling and optimization expertise, and most lab teams don't have people for that. So they change one variable at a time and follow their instincts. That burns the budget fast.

Trellis is that cohort of missing specialists, in a single platform.

1) What it does

A scientist says what they want to improve and what they're allowed to change. Trellis picks the most informative experiments to run next, hands over a plan they can execute at the bench, takes the results back in whatever shape the instruments produced them, fits a model, and proposes the next set. Then it loops.

The scientist keeps working the way they already do. They still run everything by hand. Trellis owns the choosing and the learning.

The product is a chat interface plus a canvas of editable cards holding the campaign's state. Chat is where things get decided. The cards are the durable record you can go back and change.

2) Four stages

Design works out which experiments to run. The math for choosing well is established and has been for decades, but applying it takes a specialist most teams don't have on staff. Design doesn't just hand back a list of runs. It reports which method it chose, what that design can and can't tell you afterwards, and where it had to compromise to fit the budget.

Protocol turns those runs into something a person can execute at the bench, along with a sheet to record results on. A plan names five things to vary. A real run has two hundred that matter: who ran it, which batch of raw material, whether someone was thirty minutes late to a step. Protocol is a logfile for a physical experiment. It pins what can be pinned and records what couldn't, so the things nobody named don't quietly bias the next model.

Ingest takes the files that come back and turns them into one clean dataset. This is uglier than it sounds: every instrument exports differently, nothing arrives in a standard shape, and a single run can produce spreadsheets, machine exports, and scanned paper. The output carries a receipt showing where each number came from, what got excluded, and what the data can't account for.

Model fits the results, ranks the most promising conditions to try next, and hands them back to Design. It's built to report what the evidence supports rather than what would be convenient. When there isn't enough signal to draw a real conclusion, it says so instead of producing a confident looking answer that sends the next round somewhere useless.

3) How it's built

Four agents, one per stage, each with its own tools and its own domain knowledge. They hand off in sequence, and every handoff needs a human to approve it, because each one commits to something expensive that can't be undone.

They don't share a conversation. State moves between them as versioned, schema validated facts and published artifacts, so each agent starts clean with only what it needs. Every turn runs as a fresh process and nothing durable lives in agent memory, which means a campaign survives a crash and resumes mid flight.

It runs on AWS on fully managed services: a serverless control plane, queue driven command execution, and a reconciler that repairs anything the queue drops.

4) What it isn't

It doesn't run the lab. It's not a system of record for experiments, it doesn't manage inventory or scheduling, and it never touches the physical equipment. It picks the experiments and learns from what comes back. The bench stays human.

Application Brief

5) Who uses it, and where it stops

Users: process development scientists — the principal scientist holding the whole campaign picture, and the wet-lab scientists running experiments who want to model results immediately.

In scope: the process-development stage, after a strain is available. Design, protocol planning, data canonicalization, modeling, and active learning.

Out of scope, deliberately: lab logistics, experiment tracking, and ELN. Trellis is not a system of record for the bench and does not execute bench work.

Product principles that constrain the architecture: trust and transparency are first-class; outputs must be sane rather than garbage-in-garbage-out; flows are stepwise and editable; nothing silently mutates accepted scientific context. Protocol additionally aims to progressively replace LLM-generated guidance with validated company-specific knowledge.

6) System shape

Five repositories. One platform, four domain modules:

Component	Owns
Platform (trellis)	Shared control plane, generic module-agent harness, generic MCP

Component	Owns
	host, global-context service, React frontend, local runtime, observability workbench
trellis-doe	Which experiments to run — design families, feasibility, diagnostics, capture schema
trellis-protocol	How they are run — procedure, capture sheet, deviation and pinning manifest
trellis-data	What canonical data exists — inventory, contracts, guarded pipelines, coverage, provenance, audit
Modeling	What the evidence says — screening inference, model fitting, optimization result validation

The four domain modules are independently installable Python packages. They do not import each other or the platform. They communicate only through versioned context keys and serialized artifacts. The modeling module is a thin contract-and-statistics layer over a session-oriented scientific runtime.

Runtime shape. The same handler package deploys as two Lambdas: a 29-second API Lambda serving 19 browser routes, and a long-running worker handling SQS FIFO, DynamoDB Streams, and scheduled reconciliation, so agent turns and card projections aren't cut off by the API Gateway deadline. Each domain service runs an agent image (the shared harness bound to that module's package, on Bedrock AgentCore) and an MCP image (the shared host bound to that module's tool factory, on EC2).

A turn, end to end. Browser request → verified customer identity → idempotent durable command persisted → enqueued → worker claims it and invokes the module runtime → the harness resolves the installed plugin, connects module/context/artifact MCPs, and runs the Claude Agent SDK tool loop → long operations publish durable progress messages → exactly one validated **module-agent/v2** result is written → the control plane projects messages, questions, artifacts, and any handoff → a reconciler sweep repairs anything the queue or stream missed.

7) Control flow and state

Where the model drives, and where it doesn't

- **Campaign level — hardcoded.** A fixed service registry defines entry services, transitions, and required completion artifact kinds. The model does not choose what runs next.
- **Handoffs — human-gated.** A validated handoff activates the next numbered cycle only after explicit user approval. Irreversible scientific decisions sit behind a person.
- **Within a cycle — model-driven.** One agent per module decides which tools to call and when to stop, guided by that module's Skills.
- **Topology — decentralized handoffs, not a coordinator.** Peers transfer execution; no central synthesizer. A new cycle gets a fresh conversation seeded from the public transcript only; scientific state travels through versioned context keys and artifacts, never conversation.

An internal review against the Anthropic/OpenAI/Google/Databricks agentic-design literature concluded this rung on the complexity ladder is correct — four scientific domains with large, semantically overlapping tool surfaces hit both documented triggers for splitting one agent into many. The identified problems are execution-level, not architectural.

The state model

Every turn is a fresh process. Durable state lives in global context (versioned JSON facts per campaign, with immutable value contracts owned by the first writer) and in the artifact store. The agent is stateless and horizontally scalable. This is the architecture's central strength. The modeling module is the one place it doesn't hold; its runtime keeps a cached session per namespace.

Six identifier layers coexist and are deliberately not interchangeable: platform artifact IDs (durable, published), durable context values, in-session workspace object IDs, the fitted model bundle, report IDs (evidence, not publication), and local file paths (not artifacts until published). Keeping these distinct is correct and is a real source of cognitive load.

8) Stack and scale

- **Cloud:** AWS. Lambda, API Gateway v2, SQS FIFO, DynamoDB + Streams, S3, Bedrock AgentCore, ECR, EC2 (MCP hosts), Cognito. Terraform, per-service state roots.
 - **Models:** LiteLLM → Gemini in the deployed path ([gemini-3.6-flash](#) default). Local development can also use Bedrock Claude, GPT-OSS 120B, DeepSeek V3.2, or GPT-5.6 Sol.
 - **Agent runtime:** Claude Agent SDK; module Skills delivered as SDK plugins.
 - **Tools:** MCP throughout. Global context exposes 6 tools; DoE 6; ingestion ~24; modeling 4 of its own plus the merged scientific tool surface.
 - **Scientific compute:** JAX. CPU-first; GPU is opt-in and Linux-only. Heavy fitting calls are serialized and capacity-gated.
 - **Frontend:** React/Vite, one ordered conversation plus server-declared card specs, cursor-based polling over bounded SSE batches.
 - **Code size:** ~82 authored Python files plus ~119 TS/TSX in the platform. [aws_control_plane.py](#) alone is ~6,994 lines and ~170 functions.
 - **Tests:** ~150 files across five repos — platform 71, DoE 26, Data 24, Modeling 24, **Protocol 5**. All unit or golden tests over deterministic code.
 - **Deployment status:** development Terraform roots only; no CI promotion pipeline exists. **[VERIFY]** — confirm whether anything is customer-facing today.
-

Appendix A — Component reference

Control plane. Owns browser authorization, campaigns and cycles, durable command admission, queue dispatch, turn-result projection, questions, uploads and artifact listing, handoff validation, card proxying, and recovery. Does not own scientific reasoning or module tool behavior. A service registry defines entry services, aliases, transitions, labels, and required completion artifacts.

Module agent (harness). Turns one durable command into one structured result. Resolves the installed plugin from trusted configuration, exposes its router and Skills, connects module/context/artifact MCPs, runs the Agent SDK, publishes durable progress, records the result. Does not choose tenant identity, runtime targets, allowed transitions, or completion policy. Each command is a fresh process; a cycle resumes its S3 session store from immutable gzip chunks under a conditional manifest, with a content-addressed checkpoint as a derived acceleration

layer. Liveness uses a 10-minute silence detector and a 20-minute no-progress-fingerprint detector rather than a fixed budget.

MCP host. Generic host loading one module's MCP factory over streamable HTTP, adding artifact transfer, card discovery, rendering and saving. Enforces the trusted namespace and card/artifact ownership boundary without knowing scientific meaning. `get_tool_help` provides progressive disclosure for tool schemas over a size limit. The modeling binding is the documented exception — see below.

Global context. Versioned JSON facts for one campaign, six tools, gzipped values in DynamoDB with current/history/contract row types. A contract is an immutable campaign-local value schema — the first writer owns the shape and a later writer cannot silently change it. `list_context_keys` is the progressive-disclosure index; `search_context` is lexical TF-IDF, not semantic.

Frontend. React/Vite. Cognito or fixed local sign-in, campaign and cycle navigation, one ordered conversation, validated question components, uploads, module-owned canvas tabs, generic card rendering and editing. Does not infer routing, repair backend identifiers, or compute domain results. Cursor-based finite replay polling with bounded exponential backoff and per-cursor quarantine of malformed events. Model-authored raw HTML is never enabled in chat.

Modeling. Contributes four contract-and-statistics tools of its own — screening analysis, screening effects, brief preparation, and optimization-result validation — merged with the scientific runtime's own tool registry, with duplicate names rejected by test. Eight durable `modeling.*` keys; seven read-only cards plus one editable question/route card. Three flows: screening inference, model fitting, and optimization (restricted to accepted models with a fitted result). The hosted factory adds per-namespace session caching, locking, S3 rehydration, and snapshot-after-every-call.

Local runtime. Starts the real frontend, control plane, global-context MCP, module-agent turn function, module MCP/card hosts, and all four editable domain packages, with Moto standing in for DynamoDB, Streams, SQS, and S3 and LiteLLM sending real calls to Gemini. Not a second implementation — the same production entry points behind local adapters.

Observability workbench. A separate debug-only supervisor around the real stack, with approval gates at every captured boundary (SDK↔model, LiteLLM↔provider, SDK result↔harness, context MCP, card/upload access, frontend↔control plane), an APM waterfall, a full LLM-trace reader with provider-translation comparison, hot reload, and named immutable savepoints

restorable into a fresh descendant run. Unusually strong for a team this size — and local only.

Infrastructure. Per-service Terraform state roots; the DoE root additionally owns the shared control-plane table, Lambda, API Gateway, FIFO queue, stream mappings, and reconciler schedule, coupling one domain's lifecycle to the platform's.